

Chapter 6 Exam Notes: Deadlocks

Modern Operating Systems Study Notes

Contents

1	Scope	2
2	Resources	2
2.1	Resource	2
2.2	Preemptable vs Nonpreemptable	2
3	Deadlock Definition	2
4	Four Necessary Conditions	2
5	Resource-Allocation Graph	2
6	Ostrich Algorithm	3
7	Deadlock Detection	3
7.1	Single Resource of Each Type	3
7.2	Multiple Resources of Each Type	3
8	Recovery	3
9	Deadlock Avoidance	4
9.1	Safe State	4
9.2	Unsafe State	4
9.3	Banker's Algorithm	4
10	Deadlock Prevention	4
10.1	Attack Mutual Exclusion	4
10.2	Attack Hold and Wait	4
10.3	Attack No Preemption	4

10.4 Attack Circular Wait	5
11 Communication Deadlock	5
12 Livelock and Starvation	5
13 Math-Based Problems	5
13.1 Deadlock-Free Identical Resources	5
13.2 Banker Complexity	5
13.3 Detection Matrix Steps	5
13.4 Safety Sequence	6
14 Classic Scenario Patterns	6
14.1 Traffic Gridlock	6
14.2 Database Account Locks	6
14.3 Ethernet Collision	6
14.4 Producer-Consumer Wrong Lock Order	6
15 Likely Questions with Answers	6

1 Scope

Exam focus: All of Chapter 6 is important. Study concepts, definitions, graph/model questions, deadlock detection, avoidance, prevention, recovery, livelock/starvation, and math-based resource problems.

2 Resources

2.1 Resource

A **resource** is anything a process needs to use: printer, file, lock, memory block, tape drive, database record, or message buffer.

2.2 Preemptable vs Nonpreemptable

- **Preemptable:** can be taken away safely, e.g., CPU, memory page after saving state.
- **Nonpreemptable:** cannot be taken without causing failure, e.g., printer mid-job, mutex, tape drive.

3 Deadlock Definition

A set of processes is deadlocked if every process in the set is waiting for an event that only another process in the same set can cause.

4 Four Necessary Conditions

1. **Mutual exclusion:** at least one resource is nonshareable.
2. **Hold and wait:** a process holds resources while requesting more.
3. **No preemption:** resources cannot be forcibly taken.
4. **Circular wait:** a circular chain exists where each process waits for a resource held by the next.

All four are necessary. For single-instance resources, a cycle in the resource-allocation graph is sufficient for deadlock. With multiple instances, a cycle may not be sufficient.

5 Resource-Allocation Graph

- Process: circle.
- Resource: square.

- Request edge: process to resource.
- Assignment edge: resource to process.

6 Ostrich Algorithm

Ignore deadlocks if they are rare and recovery/prevention cost is too high. Practical in some general-purpose systems, unacceptable in safety-critical systems.

7 Deadlock Detection

7.1 Single Resource of Each Type

Build a graph and check for cycles.

7.2 Multiple Resources of Each Type

Given:

- E : existing resource vector.
- A : available resource vector.
- C : current allocation matrix.
- R : request matrix.

Detection algorithm:

1. Start with all processes unmarked.
2. Find an unmarked process i with $R_i \leq A$ component by component.
3. Mark it; pretend it finishes and releases resources: $A = A + C_i$.
4. Repeat until no more process can be marked.
5. Unmarked processes are deadlocked.

Math note: Componentwise comparison means every resource type must satisfy the inequality. Example: $(1, 0, 2) \leq (1, 2, 2)$ is true; $(1, 3, 0) \leq (1, 2, 2)$ is false.

8 Recovery

Preemption Take resource away if safe.

Rollback Return process to an earlier checkpoint.

Killing processes Abort one or more processes until deadlock disappears.

9 Deadlock Avoidance

9.1 Safe State

A state is **safe** if there exists at least one sequence in which all processes can finish.

9.2 Unsafe State

An unsafe state is not deadlocked yet, but it may lead to deadlock if future requests occur.

9.3 Banker's Algorithm

Before granting a request:

1. Check request does not exceed declared maximum need.
2. Check request does not exceed available resources.
3. Pretend to grant it.
4. Run safety test.
5. Grant only if resulting state is safe.

$$\text{Need} = \text{Maximum} - \text{Allocation}$$

10 Deadlock Prevention

Break at least one necessary condition.

10.1 Attack Mutual Exclusion

Make resources shareable where possible. Example: spooling printer output instead of holding physical printer directly.

10.2 Attack Hold and Wait

Require all resources at once, or require a process to release held resources before requesting more.

10.3 Attack No Preemption

Allow resources to be taken away. Works for CPU/memory, not for many devices/locks.

10.4 Attack Circular Wait

Impose total ordering on resources. Processes request resources only in increasing order.

11 Communication Deadlock

Processes wait for messages/events from each other, not necessarily physical resources. Example: A waits for B's reply, B waits for C, C waits for A.

12 Livelock and Starvation

Deadlock Processes blocked forever.

Livelock Processes keep running but make no progress.

Starvation A process waits indefinitely because others keep getting service first.

13 Math-Based Problems

13.1 Deadlock-Free Identical Resources

If p processes each need at most m identical resources, a sufficient deadlock-free condition is:

$$r \geq p(m - 1) + 1$$

Reason: worst case is each process holds $m - 1$ resources and needs one more. One extra resource lets one process finish and release resources.

13.2 Banker Complexity

With n processes and m resource classes:

$$O(mn^2)$$

Reason: comparing one process need vector costs m ; scanning processes can cost n ; marking up to n processes gives mn^2 .

13.3 Detection Matrix Steps

For each step:

1. Write current A .
2. Find a row $R_i \leq A$.
3. Mark process i .

4. Add allocation row C_i to A .
5. Continue until stuck or all marked.

13.4 Safety Sequence

If you can list processes $P_{i_1}, P_{i_2}, \dots, P_{i_n}$ such that each process's remaining need can be met after previous ones finish, the state is safe.

14 Classic Scenario Patterns

14.1 Traffic Gridlock

Deadlock-like circular blocking. Prevention: do not enter intersection unless exit space is available. This attacks hold-and-wait.

14.2 Database Account Locks

Use global lock ordering by account number to avoid circular wait.

14.3 Ethernet Collision

Repeated immediate retransmission can cause livelock. Random backoff reduces repeated collision and starvation risk.

14.4 Producer-Consumer Wrong Lock Order

If consumer locks mutex and then sleeps waiting for producer, while producer needs mutex to produce, the system can deadlock. Correct approach: wait for nonempty condition without holding the wrong lock, or use monitor/condition-variable pattern correctly.

15 Likely Questions with Answers

Question

Q1. Define deadlock and the four necessary conditions

Answer

Deadlock occurs when every process in a set waits for an event that only another process in the same set can cause. The four necessary conditions are mutual exclusion, hold and wait, no preemption, and circular wait. All four must hold for deadlock to be possible.

Question

Q2. When is a cycle in the resource graph sufficient for deadlock?

Answer

If every resource type has only one instance, a cycle in the resource-allocation graph is both necessary and sufficient for deadlock. If resource types may have multiple instances, a cycle is necessary but not sufficient because another free instance may still allow progress.

Question

Q3. How is deadlock detected using matrices?

Answer

Use the available vector A , allocation matrix C , and request matrix R . Find an unfinished process whose request row is less than or equal to A . Pretend it finishes and add its allocated resources back to A . Repeat. If no such process exists and some processes remain unfinished, those processes are deadlocked.

Question

Q4. Safe vs unsafe vs deadlocked state

Answer

A safe state has at least one order in which all processes can finish. An unsafe state has no guaranteed safe sequence, but it may still run without deadlock depending on future requests. A deadlocked state means processes are already stuck waiting for each other and cannot proceed without intervention.

Question

Q5. Banker's algorithm steps

Answer

When a process requests resources, first check that the request does not exceed its declared maximum and that resources are available. Temporarily allocate the resources. Then run the safety test to see whether some safe completion sequence still exists. If yes, grant the request; otherwise, make the process wait.

Question

Q6. How can deadlock be prevented by breaking each condition?

Answer

Mutual exclusion can be reduced by spooling or sharing resources where possible. Hold and wait can be broken by requiring processes to request all resources at once or release held resources before asking for more. No preemption can be broken by forcibly taking resources when safe. Circular wait can be broken by imposing a total ordering on resource types and requiring requests in increasing order.

Question

Q7. Deadlock vs livelock vs starvation

Answer

In deadlock, processes are blocked and cannot proceed. In livelock, processes keep changing state but still make no progress. In starvation, one process waits indefinitely because scheduling or resource allocation keeps favoring others.

Question

Q8. Explain the identical-resource formula $r \geq p(m - 1) + 1$

Answer

Suppose there are p processes and each may need at most m identical resources. The worst nondeadlocked-but-dangerous case is that every process holds $m - 1$ resources and waits for one more. That uses $p(m - 1)$ resources. If the system has one extra resource, at least one process can get its final needed resource, finish, and release resources. Therefore $r \geq p(m - 1) + 1$ guarantees no deadlock for this model.